

Finding Tree Decompositions of Small Treewidth

Yuval Itzhaki

Claude

Abstract

The treewidth of a graph measures how far it is from a tree, and many NP-hard problems can be solved in time $f(k) \cdot \text{poly}(n)$ on graphs of treewidth k once a tree decomposition of small width is at hand. This note records two classical results that produce such a decomposition, the theorem of Bodlaender and Kloks and Bodlaender's theorem, and how they are stated and proved for the word RAM of the Lax archive, so that other formalizations can use them.

1 Introduction

A *tree decomposition* of a graph is a tree whose nodes carry sets of vertices, called *bags*, such that every vertex and every edge of the graph lies in some bag and the nodes containing a fixed vertex form a connected subtree. Its *width* is the largest bag size minus one, and the *treewidth* $\text{tw}(G)$ is the least width of a decomposition of G . Dynamic programs over a decomposition of width k run in time $f(k) \cdot \text{poly}(n)$, but they need the decomposition, and finding an optimal one is itself NP-hard when k is part of the input. For fixed k it is not, and the two theorems below say how fast.

2 Graphs and Nice Tree Decompositions as Words

A tree decomposition is *nice* if its root has an empty bag, its leaves have empty bags, and every other node is one of three kinds. An *introduce* node has one child and its bag is the bag of the child plus one vertex. A *forget* node has one child and its bag is the bag of the child minus one vertex. A *join* node has two children whose bags equal its own. Nice decompositions are due to Kloks [4] and are the form that dynamic programs consume.

The word RAM reads and writes lists of natural numbers, so a graph and a decomposition are written as such lists. A graph on the vertices $0, \dots, n-1$ is the number n followed by its adjacency matrix row by row. The matrix is used rather than an edge list so that a program filling a table indexed by pairs of vertices reads an entry in one instruction. A nice decomposition is the number N of its nodes followed by N records of three numbers in an order in which every node comes after its children. A record gives the kind of the node (leaf, introduce, forget or join), a vertex (for an introduce or a forget node) and a node (the second child of a join node). The first child of a node that is not a leaf is the node just before it, and the last node is the root. Bags are not written, since they are determined from the leaves upward. Figure 1 shows an example.

Done slightly differently than in the literature, the root of a decomposition word is not required to have an empty bag. Every consumer only uses that each vertex is introduced once along each path and forgotten after its last edge, and the root's bag can be emptied by a linear number of further forget nodes.

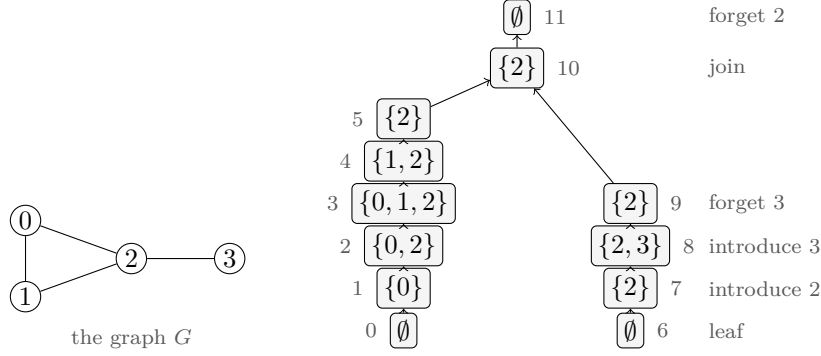


Figure 1: A graph on four vertices and a nice tree decomposition of width 2, with the root at the top. The node numbers are the positions in the word of the decomposition: each node is written as its kind, its vertex, and, for the join node 10, its second child (node 5; its first child is the node just before it).

3 The Two Theorems

Both results are stated on the word RAM of the archive as follows. One program and one constant serve every word length, every graph and every bound. They apply whenever the input fits with room for $c \cdot 2^{\ell^3}$ times a polynomial in its length, a condition checkable from the input alone, and then the program halts within the stated number of instructions. The output is either the word [0], admissible only when G has no tree decomposition of width at most k , or the number 1 followed by the word of a nice tree decomposition of width at most k . It is a relation rather than a function, since a graph has many decompositions. The bounds are polynomial in the encoded input length. The graph word has quadratic length in the number of vertices; for the improvement theorem, the input also includes the supplied decomposition, whose size is unrestricted. The formalized bounds are polynomial rather than linear in these lengths.

Theorem 1 (Bodlaender and Kloks [3]). *Given a graph G , a bound k and a nice tree decomposition of G of width at most ℓ , one can decide whether $\text{tw}(G) \leq k$ and, if so, find a nice tree decomposition of width at most k , in time $2^{O(\ell^3)} \cdot \text{poly}(N)$, where N is the length of the complete encoded input, including the supplied decomposition.*

The exported proof uses the exact graph algorithm described below. If $\ell \leq k$, it returns the supplied decomposition. Otherwise it runs the graph algorithm with bound k , independently of the supplied decomposition. Since $k < \ell$, its parameter-dependent running time is bounded by $2^{O(\ell^3)}$; reading and returning the supplied decomposition is covered by the polynomial in the full input length.

Theorem 2 (Bodlaender [2]). *Given a graph G and a bound k , one can decide whether $\text{tw}(G) \leq k$ and, if so, find a nice tree decomposition of G of width at most k , in time $2^{O(k^3)} \cdot \text{poly}(|G|)$.*

The proof uses a dynamic program over characteristics of partial decompositions, following Althaus and Ziegler [1]. Typical sequences describe the characteristics; introduce, forget and join transitions compute the tables, from which a decomposition is extracted. The vertices are inserted one at a time. Having a nice decomposition of the graph on the first i vertices, insert the next vertex into every bag. This is a decomposition of the larger graph whose width has grown by one, and the improvement step of this dynamic program brings the width back down to k , or reports that the treewidth exceeds k . The graph algorithm is proved first; the exported improvement theorem is then derived from it.

Done slightly differently than in the paper of Bodlaender, whose algorithm shrinks the graph by a constant factor and recurses with linear running time, the algorithm formalized is the vertex

by vertex wrapper above. Its running time is $2^{O(k^3)} \cdot \text{poly}$, which is what the statement asks. Compression before conversion to a nice tree keeps the sizes of intermediate decompositions polynomial in the graph size, with a parameter-dependent factor.

4 How the Proofs Are Organized

The proofs have two layers. The first proves correctness of the dynamic program on mathematical objects, covering the sequences and their laws, decompositions as rooted trees with their bridges to the archive's tree decompositions and to the word format, characteristics and their normal form, the four kinds of node, and the tables. The second proves the running time. The Lean functions of the first layer are run on a virtual machine for a first-order functional language whose semantics carries a cost, a verified compiler takes that language to bytecode written in IMP+, and the runs are transferred to the word RAM through the archive's compiler. Both theorems depend on Lean's three standard axioms only.

References

- [1] E. Althaus and S. Ziegler. Optimal Tree Decompositions Revisited: A Simpler Linear-Time FPT Algorithm. arXiv:1912.09144, 2020.
- [2] H. L. Bodlaender. A Linear-Time Algorithm for Finding Tree-Decompositions of Small Treewidth. *SIAM Journal on Computing*, 25(6):1305–1317, 1996.
- [3] H. L. Bodlaender and T. Kloks. Efficient and Constructive Algorithms for the Pathwidth and Treewidth of Graphs. *Journal of Algorithms*, 21(2):358–402, 1996.
- [4] T. Kloks. *Treewidth: Computations and Approximations*. Lecture Notes in Computer Science 842, Springer, 1994.